

# Algoritmos y Estructuras de Datos

## Vectores Unidimensionales



### Contenidos del documento:

- ❖ Declaracion e inicializacion
- ❖ Carga secuencial
- ❖ Operaciones Basicas: (Recorrido, Busqueda y Agregacion)
- ❖ Insercion y eliminacion manual
- ❖ Subvectores y Segmentacion

## Declaracion e inicializacion:

En C# Un array es un objeto de tamaño fijo que almacena elementos del mismo tipo.

- Declaración: **tipo[] nombre;**
- Inicialización: Se debe especificar el tamaño usando la palabra clave **new**.

```
// Declaración y luego inicialización

int[] edades;

edades = new int[5]; // Array de 5 enteros, inicializados en 0 por defecto.

// Declaración e inicialización inmediata con valores

string[] nombres = { "Ana", "Luis", "Marta" };
```

## Carga Secuencial:

La carga secuencial se refiere a llenar el array elemento por elemento, usualmente mediante un ciclo **for** o pidiendo datos al usuario.

```
int[] numeros = new int[3];

for (int i = 0; i < numeros.Length; i++)
{
    Console.Write($"Ingrese el valor para la posición {i}: ");

    numeros[i] = int.Parse(Console.ReadLine());
}
```

## Recorrido:

Para visitar cada elemento, usamos **foreach** (lectura) o **for** (lectura/escritura).

```
int contadorPares = 0;

foreach (int n in numeros)
{
    if (n % 2 == 0) contadorPares++;
}
```

## Busqueda (Secuencial):

Consiste en comparar un valor objetivo con cada elemento hasta encontrarlo.

```
int buscado = 10;
int posicion = -1;

for (int i = 0; i < numeros.Length; i++)
{
    if (numeros[i] == buscado)
    {
        posicion = i;
        break; // Salimos al encontrar la primera coincidencia
    }
}
```

## Agregación (Suma y Promedio):

```
int suma = 0;

for (int i = 0; i < numeros.Length; i++) suma += numeros[i];

double promedio = (double)suma / numeros.Length;
```

## Inserción y Eliminación Manual (Desplazamiento)

Como los arrays en C# tienen un tamaño **estático**, "insertar" o "eliminar" requiere mover los elementos manualmente para no dejar huecos o perder datos.

### Inserción por desplazamiento

Para insertar un valor en la posición **k**, debemos desplazar los elementos desde **k** hacia la derecha.

```
// Insertar el valor 99 en el índice 2

int indiceInsertar = 2;

int nuevoValor = 99;

// Desplazamos de derecha a izquierda para no sobrescribir
for (int i = numeros.Length - 1; i > indiceInsertar; i--)
{
    numeros[i] = numeros[i - 1];
}

numeros[indiceInsertar] = nuevoValor;
```

## Eliminación por desplazamiento

Para eliminar el elemento en **k**, desplazamos los elementos de la derecha hacia la izquierda.

```
int indiceEliminar = 1;

for (int i = indiceEliminar; i < numeros.Length - 1; i++)
{
    numeros[i] = numeros[i + 1];
}

numeros[numeros.Length - 1] = 0; // Opcional: "limpiar" el último espacio
```

## Subvectores y Segmentación:

C# facilita extraer partes de un array mediante **Slices** (segmentos) o el uso de **Span<T>**.

También podemos usar el operador **..** para obtener una porción del vector.

```
int[] original = { 0, 10, 20, 30, 40, 50 };

// Obtener del índice 1 al 3 (el 4 queda excluido)

int[] subVector = original[1..4]; // Resultado: { 10, 20, 30 }

// Usando Span (más eficiente en memoria, no crea una copia física)

ReadOnlySpan<int> segmento = original.AsSpan(2, 3); // Desde índice 2,
toma 3 elementos
```

La segmentación con **[1..4]** crea una copia del array original, mientras que **Span<T>** es una vista de los mismos datos en memoria.

